

POS Tagging with Recurrent Neural Networks: A Comparative Study

Abstract

In our project, we used the POS 1 dataset to perform Part-of-Speech (POS) tagging with four neural network models, which are, Simple Recurrent Neural Network (RNN), a Long Short-Term Memory network (LSTM), a Gated Recurrent Unit network (GRU), and a Bidirectional LSTM (BiLSTM). We explored the dataset, preprocessed the text, and built the models using TensorFlow/Keras. After tuning the models, we evaluated them using accuracy, F1-score, confusion matrices, and classification reports. The BiLSTM model performed best, achieving 90.3% accuracy and a 0.897 weighted F1-score.

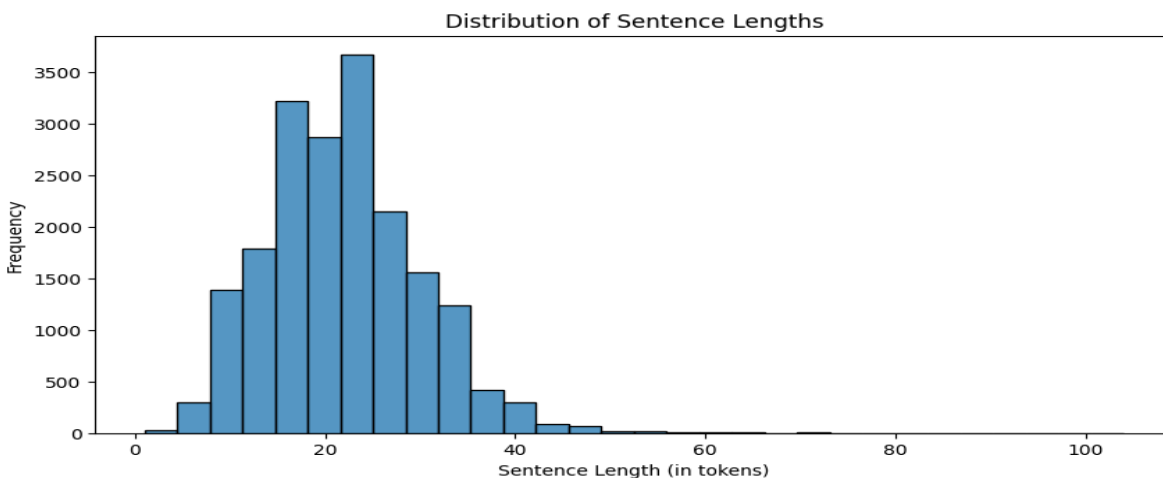
Introduction

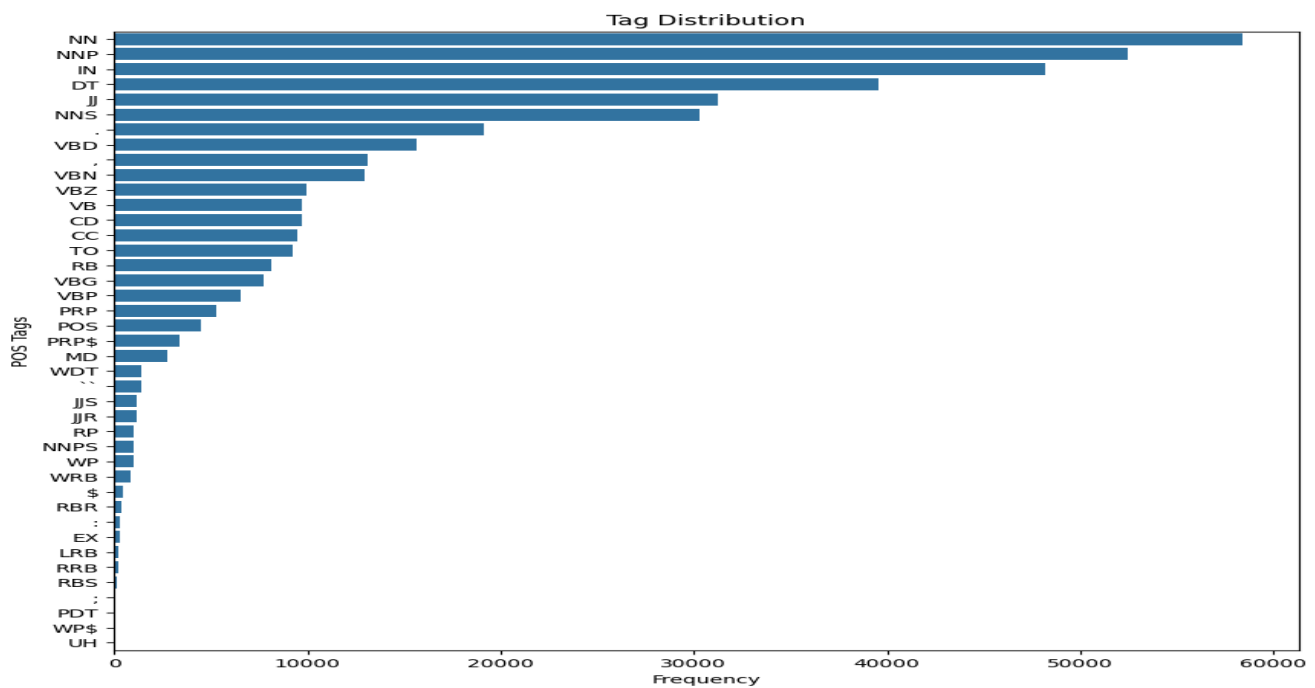
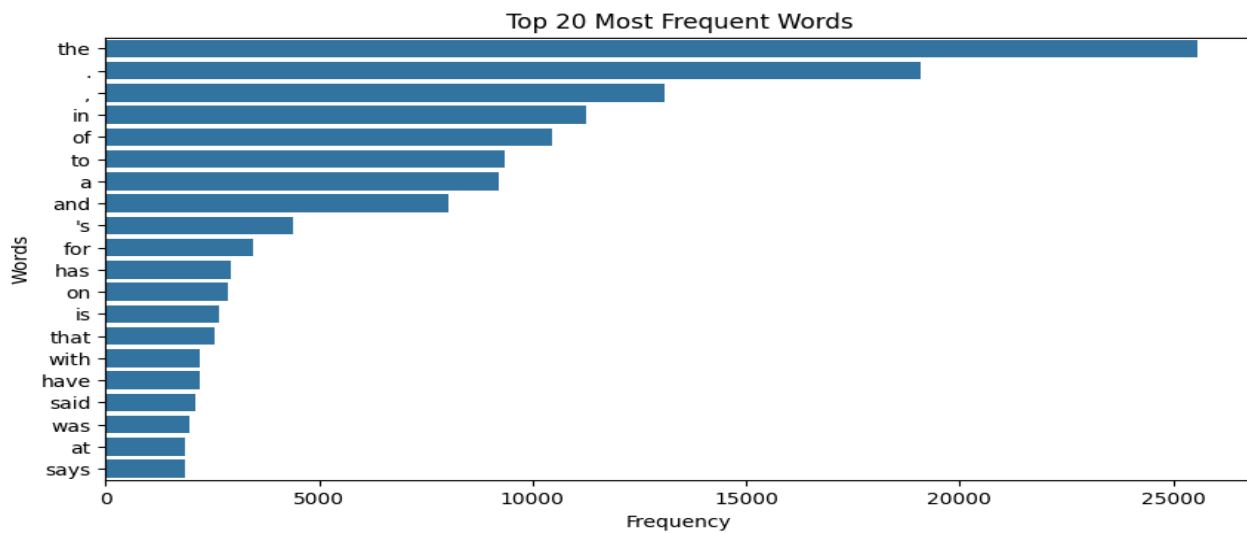
POS tagging is an important task in Natural Language Processing. It assigns parts-of-speech tags to words in a sentence. It helps with tasks like text analysis and translation. In this project, we were assigned the POS 1 dataset to compare four Recurrent Neural Network models, which are Simple RNN, LSTM, GRU, and BiLSTM. Our goal was to find the best model for POS tagging by testing and comparing each model's performance on the dataset.

Methodology

Data Analysis

The POS 1 dataset is split into two parts: training and testing sets. The training had 19,183 sentences, and the Testing had 4,796 sentences. We used the training set to analyze the data and train models, keeping the test set for final evaluation. The dataset has three columns: Sentence #, Sentence, and POS tags. The first column indicates the serial number of a sentence, and the second column is the sentence itself. The third is the POS tags of that particular sentence. There was no missing data in the dataset. Upon further analysis, there was a mismatch between the number of tokens and the number of POS tags for 3 sentences. There were 41 unique POS tags, with 418,968 total tags in the training set. Tags like 'NN', 'NNP', and 'IN' were frequent. On the other hand, 'WPS', 'UH' were very infrequent. We created bar plots to show the sentence length distribution, tag distribution, and the most frequent words, which helped us understand the data.





- ➔ Dropped training data with mismatching number of tokens and Pos tags
- ➔ Converted POS tag strings to lists.
- ➔ Used Keras Tokenizer to convert words to numbers.
- ➔ Encoded POS tags as numbers using Label Encoder.
- ➔ Padded all sentences to 100 words for consistent input.
- ➔ Created a stratification excuse group for too infrequent data.
- ➔ Used GloVe word embeddings with 300 dimensions to improve model performance.

We split the training data into 80% for training and 20% for validation to tune the models.

Model Architectures

We built four models using TensorFlow/Keras:

Input Layer :

Each model had the same input layer, which created dense vectors using pre-trained GloVe embeddings. The trainable parameter was set to False to keep the embedding matrix frozen.

Recurrent Layers:

Simple RNN:

Two stacked SimpleRNN layers with 32 units each and return_sequences=True.

LSTM:

Two stacked LSTM layers with 32 units each and return_sequences=True.

GRU:

Two stacked GRU layers with 32 units each and return_sequences=True.

Dropout (0.2) is applied between all the recurrent layers to prevent overfitting.

Output Layer:

Each model had an output layer, where a fully connected dense layer uses the Softmax Activation function. It uses the unit size of the number of classes of tags.

Hyperparameter Tuning

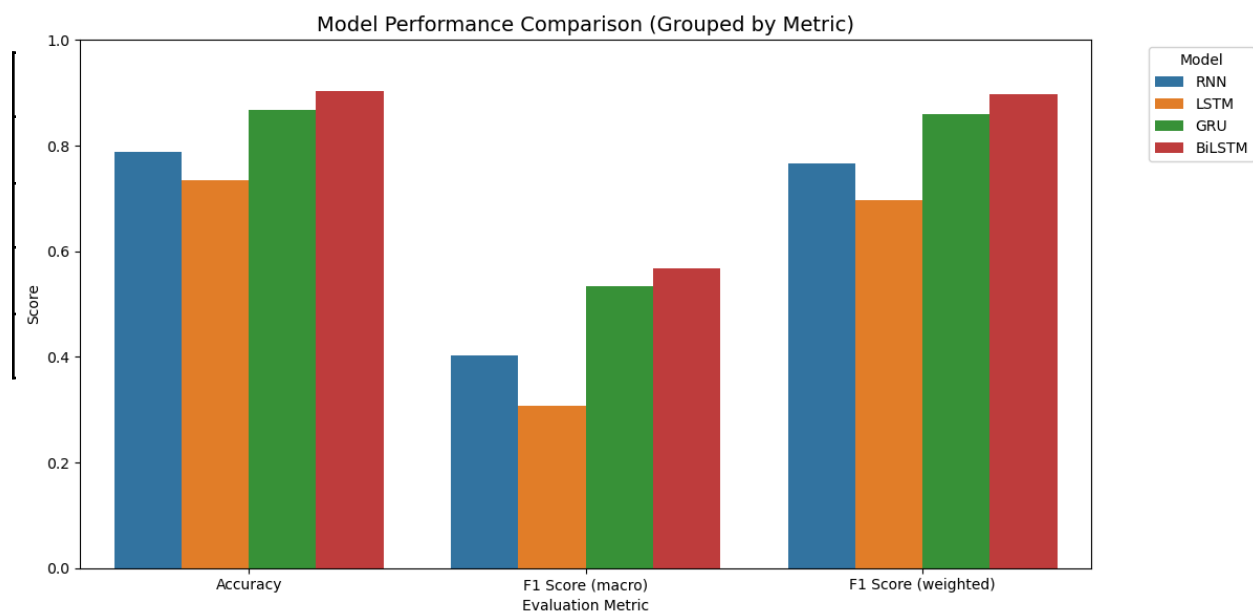
We tuned the models by testing and used the Adam optimizer:

- Learning rates: 0.001, 0.0005, 0.0001, with learning rate reduction on plateau by 0.5
- Batch sizes: 32, 128, 256.
- Epochs: Up to 50, with early stopping if no improvement after 3 epochs.

The best settings were a learning rate of 0.001, a batch size of 256, and around 15 epochs.

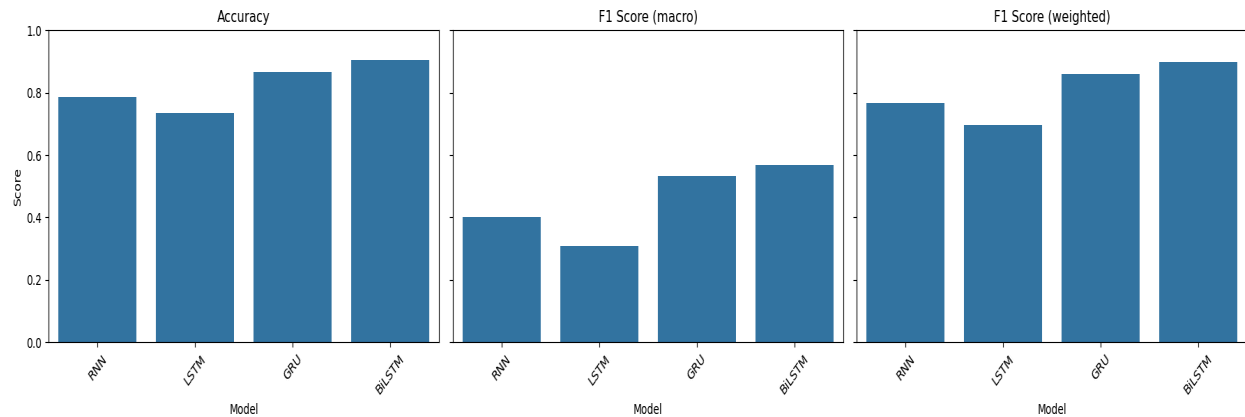
Results

We evaluated the models on the test set using accuracy, macro F1-score, weighted F1-score, confusion matrices, and classification reports. The results are shown below:

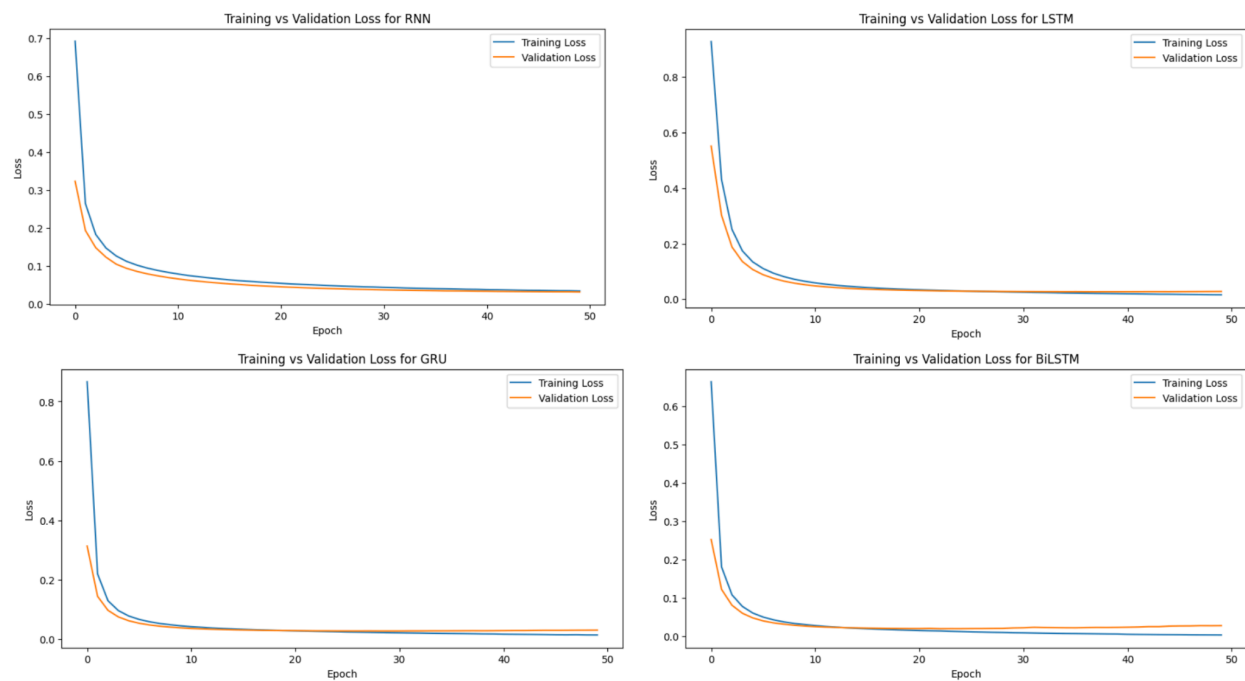


Individual Model Comparison Metrics:

Model Comparison by Individual Metrics



Training and Validation Loss Curve:

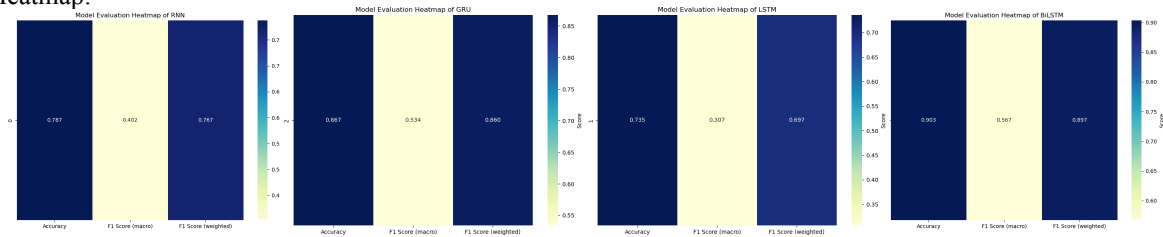


These curve indicates there's almost no sign of overfitting. And the flat line at the end suggests we have reached an optimal point of accuracy achievable by this model.

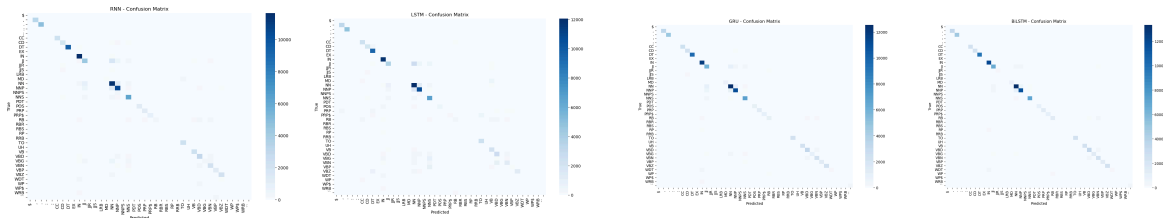
Best model: The BiLSTM model performed best, with 90.3% accuracy and a 0.897 weighted F1-score. It scored 0.57 on F1 Score(macro), which means it has some difficulties with some classes. But the model still performed better than the other 3 models.

Other Models: The LSTM was the weakest due to being easier to overfit. Simple RNN generalized better with its less sensitive model. And GRU showed effectiveness, but with unidirectionality, it could not beat BiLSTM. Confusion matrices showed that most errors were between similar tags like NN and NNS. Rare tags like WP\$ had lower performance due to fewer examples. A plot of validation accuracy showed BiLSTM converged faster and better than the others.

Heatmap:



Confusion Matrix:



Conclusion

The BiLSTM model was the best for POS tagging because it uses context from both directions of the sentence. The Simple RNN struggled with long sequences. LSTM and GRU performed moderately well. This project shows that advanced RNN models like BiLSTM are highly effective for POS tagging.

Although BiLSTM is highly effective, every model has limitations and struggles to identify the classification between very similar tags. The dataset also has a very low frequency for some tags, resulting in limited performance for less frequent tags. Also, capturing the sequence helped GRU and BiLSTM to perform better. But it has limitations; redefined models like BERT can be better with stronger context capture.

There's always room for improvement in feature engineering and fine-tuning hyperparameters. To further improve the results, we can augment data for less frequent tags, design each model with unique sensitivity, combine multiple models together, and so on.

Overall, this project provided an insightful comparison between multiple models and helped us have a better understanding of how recurrent neural networks work.

References

1. J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in Proc. EMNLP, 2014, pp. 1532–1543.
2. F. Chollet et al., "Keras," <https://keras.io>, 2015.
3. M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," <https://www.tensorflow.org>, 2015.